TP4 : Gestionnaires de paquets (uv) et outils CLI avec argparse

Enseignant : Abderrahmane Kasmi | **Public :** 1ère année Cycle Ingénieur ISEN

1 Introduction

Ce TP a pour objectif de vous apprendre à initialiser et structurer un projet Python moderne, à gérer un environnement virtuel et des dépendances externes avec l'outil `uv`, ainsi qu'à concevoir un outil en ligne de commande (CLI) en utilisant le module standard `argparse`.

Objectifs du TP

À l me de ce TP, vous serez capable de :

- Installer et configurer le gestionnaire de paquets `uv`.
- Initialiser un projet Python et gérer ses dépendances avec `uv`.
- Déclarer et exploiter la bibliothèque tierce `colorama` pour formater des sorties terminal.
- Comprendre le rôle des fichiers de configuration `pyproject.toml` et de verrouillage `uv.lock`.
- Développer un outil CLI avec `argparse` utilisant des arguments optionnels.
- Exécuter le programme dans son environnement isolé à l'aide de la commande `uv run`.

2 Exercice — Outil de sécurité CLI `secuguard`

2.1 Description du projet

Vous devez développer l'application `secuguard`, une boîte à outils en ligne de commande dédiée à la sécurité des mots de passe. Elle permet :

1. De **générer** un mot de passe sécurisé et personnalisable selon divers critères.
2. D'**analyser** la robustesse d'un mot de passe existant et d'en afficher le diagnostic coloré dans le terminal.

2.2 Installation de `uv`

Si `uv` n'est pas encore installé sur votre machine, exécutez la commande correspondant à votre système d'exploitation dans votre terminal :

Sur Linux et macOS :

```
curl -LsSf https://astral.sh/uv/install.sh | sh
```

Sur Windows (PowerShell) :

```
powershell -ExecutionPolicy ByPass -c "irm https://astral.sh/uv/install.
    ps1 | iex"
```

Alternative via `pip` :

```
pip install uv
```

Vérifiez ensuite l'installation avec la commande `uv -version`.

2.3 Initialisation du projet avec `uv`

Initialisez le projet et ajoutez la dépendance `colorama` via le gestionnaire de paquets `uv` :

```
uv init tp4_secuguard
cd tp4_secuguard
uv add colorama
```

2.4 Architecture à créer

Créez l'arborescence standard suivante dans votre projet :

```
tp4_secuguard/
|-- pyproject.toml
|-- uv.lock
|-- README.md
‘-- src/
    ‘-- secuguard/
        |-- __init__.py
        |-- generator.py
        |-- analyzer.py
        ‘-- main.py
```

2.5 Spécifications détaillées des fonctions

2.5.1 Module `generator.py` — Génération de mot de passe

Définissez la fonction principale suivante :

```
def generer_mot_de_passe(longueur: int = 12, avec_chiffres: bool = True,
    avec_symboles: bool = True) -> str
```

Consignes d'implémentation :

— Utilisez les ensembles de caractères fournis par le module standard `string` (`string.ascii_letters`, `string.digits`, `string.punctuation`).

— Constituez l'alphabet global A selon la logique suivante :

$$A = \text{ascii_letters} \cup (\text{digits if avec_chiffres}) \cup (\text{punctuation if avec_symboles})$$

— Utilisez la fonction `random.choice()` pour sélectionner aléatoirement N caractères dans cet alphabet, où $N =$ longueur.

2.5.2 Module `analyzer.py` — Analyse et coloration

Définissez la fonction d'évaluation et de mise en forme :

```
def analyser_mot_de_passe(mot_de_passe: str) -> str
```

Consignes d'implémentation :

— **Calcul de la robustesse :**

  - Si longueur ≥ 12 et que le mot de passe contient des lettres, des chiffres et des symboles $\Longrightarrow$ **Niveau FORT**.
  - Si longueur ≥ 8 avec au moins deux types de caractères différents $\Longrightarrow$ **Niveau MOYEN**.
  - Sinon $\Longrightarrow$ **Niveau FAIBLE**.
- **Coloration avec `colorama` :** Importez `from colorama import Fore, Style, init` et appelez `init(autoreset=True)` au début du module.
  - Vert (`Fore.GREEN`) pour le niveau **FORT** (ex : `Fore.GREEN + "[FORT] Mot de passe tres solide."`).
  - Jaune (`Fore.YELLOW`) pour le niveau **MOYEN**.
  - Rouge (`Fore.RED`) pour le niveau **FAIBLE**.

2.5.3 Module `main.py` — Interface `argparse`

Créez la CLI à l'aide d'un parser d'arguments unique (sans sous-parseurs).

Argument / Option	Configuration `argparse` attendue
`-check` (`-c`)	Option acceptant une chaîne de caractères (`str`) : le mot de passe à analyser. Si cette option est fournie, le programme lance l'analyse.
`-length` (`-l`)	Option de type `int`, valeur par défaut $= 12$. Longueur du mot de passe à générer.
`-no-digits`	Option avec `action="store_false"`, destination `avec_chiffres`. Exclut les chiffres du générateur.
`-no-symbols`	Option avec `action="store_false"`, destination `avec_symboles`. Exclut les caractères spéciaux du générateur.

Comportement du `main()`

- Si `args.check` est spécifié $\Longrightarrow$ exécuter `analyser_mot_de_passe(args.check)` et afficher le résultat coloré.
- Sinon $\Longrightarrow$ exécuter `generer_mot_de_passe(args.length, args.avec_chiffres, args.avec_symboles)` et afficher le mot de passe généré.
- Le point d'entrée doit être protégé par `if __name__ == "__main__"`.

2.6 Exemples d'exécution avec `uv run`

1. Génération par défaut

```
uv run python -m secuguard.main
```

Sortie : `Mot de passe genere : aK9#mP2!xL8q`

2. Génération personnalisée ($N = 16$, sans symboles)

```
uv run python -m secuguard.main -l 16 --no-symbols
```

Sortie : `Mot de passe genere : F8k92A7x1M9p4L0z`

3. Analyse d'un mot de passe faible

```
uv run python -m secuguard.main -c "12345"
```

Sortie (en texte rouge) : `[FAIBLE] Mot de passe trop court et prévisible.`

4. Analyse d'un mot de passe fort

```
uv run python -m secuguard.main --check "P@ssw0rd2026!Secu"
```

Sortie (en texte vert) : `[FORT] Mot de passe tres solide.`

2.7 Livrables

Déposez une archive `.zip` du projet `tp4_secuguard/` contenant :

- Les fichiers source sous `src/secuguard/`.
- Les fichiers de gestion de projet `pyproject.toml` et `uv.lock`.
- Un fichier `README.md` expliquant les étapes d'installation et de lancement avec `uv`.